LLARS - Script Creator Library

Current Creator API Recipes

Current for LLARS v2

Scope - This guide documents the public LLARS project as it operates in v2. Follow the shared framework and the current GUI/API behavior described here.

Contents

1. How to Use This Library
2. Config Reads
3. RunCount Recipe
4. Click Recipes
5. Keyboard Recipes
6. Status and Diagnostics
7. Sleep and Runtime Recipes
8. Optional Features
9. Managed Timer Recipes
10. Timer Script Recipe
11. MultiTimer Recipe
12. Single and Multi-Color Detection
13. Prevent Repeated Pixel Triggers
14. Dynamic Color Targets
15. Advanced Detection State
16. Common Action Combinations
18. Conversion Mappings
19. Patterns to Avoid
20. Creator API Cheat Sheet

1. How to Use This Library

Copy a recipe into the script editing region of a current template and replace section names with names that exist in that script Config.ini. These recipes assume the normal LLARS bootstrap has already called LLARS_Initialize().

Input rule - All public script/template physical input goes through LLARS shared input APIs. Recipes below intentionally avoid raw Send, Click, MouseClick, MouseMove, direct AHI calls, and direct Win32 input injection.

2. Config Reads

```
name := LLARS_ConfigReadString("Options", "name", "Default")
count := LLARS_ConfigReadInteger("Options", "count", 1, 1, 100)
threshold := LLARS_ConfigReadNumber("Options", "threshold", 0.5, 0, 1)
enabled := LLARS_ConfigReadBool("Feature")
mode := LLARS_ConfigReadChoice("Options", "mode", ["Fast", "Normal", "Careful"], "Normal")
hotkey := LLARS_ConfigReadHotkey("Action Hotkey")
color := LLARS_ConfigReadColor("Target Color")
```

3. RunCount Recipe

```
Start:
LLARS_RunCount(Func("RunTask"))
return
```

```
RunTask(ctx)
{
    LLARS_SetStatus("Working", "Run " . ctx.Iteration . "/" . ctx.Total)
    if (ctx.IsFirst)
        LLARS_LogEvent("Run started")

    LLARS_Click("Action Location")
    LLARS_Sleep("Action Delay", ctx.IsLast)
}
```

4. Click Recipes

```
LLARS_Click("Action Location")
LLARS_Click("Action Location", "right")
LLARS_RedClick("Action Location")
LLARS_RedClickRetry("Action Location")

; Already-resolved client target:
LLARS_ClickPoint(foundX, foundY, "left", "Detected target")
```

Use RedClick variants when the activity expects RuneScape red-click confirmation. Retry behavior is intentionally explicit.

5. Keyboard Recipes

```
LLARS_PressKey("Space")
LLARS_PressKey("Escape")
LLARS_PressHotkey("Action Hotkey")
```

6. Status and Diagnostics

```
LLARS_SetStatus("Working", "Bank")
LLARS_LogEvent("Bank cycle", "Opened preset")
LLARS_SetDetection("Target", "Searching")
LLARS_Recovered("Target reacquired", "Nearby scan")

if (problem)
    LLARS_Warn("Script Warning", "Target was not found.", "No click was sent.", "Check the game
layout.", "", false)
```

Use LLARS_Error for a failure that stops/blocks normal progress. Use LLARS_Warn when the script can continue or wait. Use LLARS_Recovered after a meaningful automatic recovery. There is no generic logger named Log in the current Creator API; use LLARS_LogEvent().

7. Sleep and Runtime Recipes

```
LLARS_Sleep("Short Delay")
LLARS_Sleep("Final Delay", true)
LLARS_RandomSleep()
```

Configured LLARS_Sleep calls contribute to RunCount estimation. Keep timed waits in named Config.ini sections when users should be able to tune them.

8. Optional Features

```
if LLARS_ConfigEnabled("Optional Feature")
{
    LLARS_SetStatus("Working", "Optional Feature")
    LLARS_Click("Optional Action")
}
```

9. Managed Timer Recipes

```
onceID := LLARS_TimerOnce("Startup Delay", Func("DoOnce"))
repeatID := LLARS_TimerRepeat("Script Timer", Func("Tick"))

Tick()
{
    if !LLARS_RunActive()
        return
    LLARS_LogEvent("Timer", "Callback")
}

LLARS_TimerStop(repeatID)
LLARS_TimerStopAll()
```

10. Timer Script Recipe

```
Start:
if (!LLARS_StartTimerRun())
    return
SetTimer, Countdown, 1000
ScriptTimerID := LLARS_TimerRepeat("Script Timer", Func("ScriptTimer"))
return

ScriptTimer()
{
    if !LLARS_RunActive()
        return
    LLARS_SetStatus("Working", "Timer Action")
    LLARS_Click("Action Location")
}
```

11. MultiTimer Recipe

```
TimerOneID := LLARS_TimerRepeat("Timer One", Func("TimerOne"))
TimerTwoID := LLARS_TimerRepeat("Timer Two", Func("TimerTwo"))

TimerOne()
{
    if LLARS_RunActive()
        LLARS_Click("Action One")
}
```

```
TimerTwo()
{
    if LLARS_RunActive()
        LLARS_PressHotkey("Action Hotkey")
}
```

12. Single and Multi-Color Detection

```
if LLARS_PixelMatches("Pixel Coordinate", "Target Color")
{
    LLARS_SetDetection("Target", "Matched")
    LLARS_Click("Action Location")
}

matched := LLARS_PixelMatchesAny("Pixel Coordinate", ["State One", "State Two"])
```

For tolerant matching, use LLARS_PixelMatchesWithinTolerance(). For current multi-color template behavior, map type=color sections to coordinate= and action= section names.

13. Prevent Repeated Pixel Triggers

```
CheckPixel:
if !LLARS_RunActive()
    return

if LLARS_PixelMatches("Pixel Coordinate", "Target Color")
{
    SetTimer, CheckPixel, Off
    LLARS_Click("Action Location")
    SetTimer, ResetCheck, 100
}
return

ResetCheck:
if !LLARS_RunActive()
    return
if !LLARS_PixelMatches("Pixel Coordinate", "Target Color")
{
    SetTimer, ResetCheck, Off
    SetTimer, CheckPixel, 100
}
return
```

14. Dynamic Color Targets

```
if LLARS_FindColorCenterOut(["Target Color"], foundX, foundY, matchedSection, "", "Target")
{
    if LLARS_VerifyColorTarget(foundX, foundY, matchedSection, "Target")
        LLARS_ClickPoint(foundX, foundY, "left", "Target")
}
```

Advanced LLARS_Color* helpers provide shared capture, density, clustering, connected-component, signature, and tracking behavior. Prefer them to script-local capture/input plumbing.

15. Advanced Detection State

```
LLARS_SetDetection("Resource", "Searching", "", "", "Center-out scan")

if (found)
    LLARS_SetDetection("Resource", "Matched", score, minimumScore, "Stable component")
else
    LLARS_SetDetection("Resource", "Not found", 0, minimumScore, "Waiting")
```

16. Common Action Combinations

```
LLARS_SetStatus("Working", "Action")
LLARS_Click("Action Location")
LLARS_Sleep("Action Delay")
LLARS_PressHotkey("Action Hotkey")
LLARS_LogEvent("Action complete")
```

17. Developer Mode Checks

- Open Full Mode and confirm the active coordinate range appears for LLARS clicks.
- Confirm Configuration Diagnostics matches the script Config.ini dependency state.
- For detection scripts, verify the Detection row changes meaningfully rather than flooding identical events.
- Use Copy Support to verify the generated report contains the current script/input/detection state needed for support.
- Repeat a normal run with Developer Mode closed.

18. Conversion Mappings

Avoid in current public scripts	Use instead
Send, {Space}	LLARS_PressKey("Space")
Send, %configuredHotkey%	LLARS_PressHotkey("Action Hotkey")
Click / MouseClick / MouseMove	LLARS_Click / LLARS_ClickPoint / shared target helpers
ad-hoc randomized Sleep	LLARS_Sleep or LLARS_Timer*
raw PixelGetColor for configured state	LLARS_PixelColor / LLARS_PixelMatches*
generic custom logger	LLARS_LogEvent("...", "...")
script-local message-only error	LLARS_Warn / LLARS_Error with action and recovery guidance

19. Patterns to Avoid

- Raw physical input under Scripts/Templates.
- Direct AutoHotInterception calls from an activity script.
- Direct Win32 input injection from an activity script.
- Unmanaged repeating timers when LLARS_TimerRepeat fits.
- Hard-coded setup values that belong in typed Config.ini sections.
- Using Developer Mode state to decide whether automation should work.
- Writing a new local logger or support file when the shared diagnostics stream fits.

20. Creator API Cheat Sheet

Need	Use
Read config	LLARS_ConfigRead* / LLARS_ConfigEnabled / LLARS_ConfigReadRange / LLARS_ConfigReadPoint
Run loop	LLARS_RunCount / LLARS_RunActive
Click	LLARS_Click / LLARS_RedClick / LLARS_ClickPoint
Keyboard	LLARS_PressKey / LLARS_PressHotkey
Wait	LLARS_Sleep / LLARS_RandomSleep
Timer	LLARS_TimerOnce / LLARS_TimerRepeat / LLARS_TimerStop(All)
Status	LLARS_SetStatus
Log/support	LLARS_LogEvent / LLARS_Warn / LLARS_Error / LLARS_Recovered
Detection	LLARS_SetDetection / LLARS_Pixel* / LLARS_Color*
Focus gate for specialized non-input work	LLARS_WaitForRuneScape

LLARS - Script Creator Library

Config write and enumeration recipes

19. Config Write Recipe

```
newMin := 5000
newMax := 8000

if !LLARS_ConfigWrite("Timer", "min", newMin)
    return
if !LLARS_ConfigWrite("Timer", "max", newMax)
    return
```

Writes use the same script-local path resolution as LLARS_ConfigRead*. Pass scope="root" only when the setting is intentionally owned by the shared LLARS Config.ini.

20. Enumerate Config Sections

```
sections := LLARS_ConfigSections()
if (sections = "ERROR")
    return

Loop, Parse, sections, `n, `r
{
    sectionName := Trim(A_LoopField)
    if (sectionName = "")
        continue
    ; Read metadata with LLARS_ConfigRead* helpers.
}
```

This pattern is appropriate for intentionally dynamic configuration such as the MultiColor template. For fixed settings, prefer explicit typed reads instead of enumerating the file.

Creator rule - Do not use native IniRead, IniWrite, or IniDelete in public Scripts/Templates.
Use LLARS_ConfigRead*, LLARS_ConfigWrite, and LLARS_ConfigSections so path/error behavior stays shared.